Bayesian Networks Code Report

Simon Stronks S1137749

April 3, 2026

Contents

1 Introduction

In this report, I will explain the implementation and the evaluation of a Bayesian Network system. I wrote this as 1 complete, integrated report covering Variable Elimination (VE), MAP inference, and Expectation-Maximization (EM). In this report, I will discuss algorithmic complexity and heuristic choices. I also mentioned how I used the formative feedback to improve the code. There are 3 parts:

- **Part 1: Variable Elimination (VE)**
- **Part 2: MAP inference (MAP)**
- **Part 3: Expectation-Maximization (EM)**

I run the complete pipeline from `run.py`. Logs with all results are written to `log_endorisk.txt`.

In the final code version, the active workflow is focused on the standard text BIF files used in this project setup, with `endorisk_new.bif` as the Part 3 model. I also compared the learned Endorisk model with the original one using a separate comparison script, so we can see the actual parameter changes.

2 The Background: Project Setup

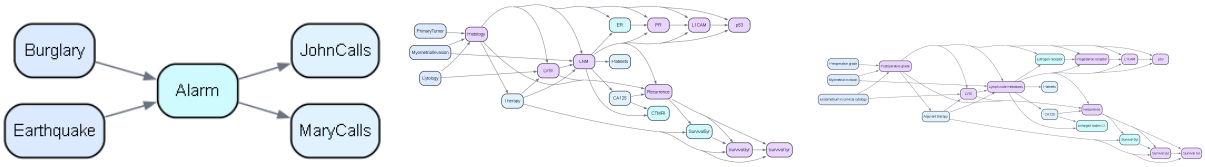
The project is structured so that all main outputs can be generated from one run command. This made it much easier for me to compare results after each heuristic change.

2.1 Project setup

- Run file: `run.py`
- Output log file: `log_endorisk.txt`
- Part 1 and Part 2 use the `earthquake.bif` network
- Part 3 network: selected in `run.py`: `endorisk_new.bif`
- Part 3 data: `simulation_data_hid_names.dat`

2.2 Visualisation network files

I wanted to visualize the .bif files better, so with the help of Graphviz I generated visual depictions of the networks.



(a) Earthquake network (b) Endorisk Network (c) Endomcancer Network

Figure 1: Visual depictions of the Bayesian networks.

2.3 Note on File Formats

During development I tested multiple file variants, but in the final version I removed the XML parsing path from `read_bayesnet.py` and kept one robust text-BIF parsing flow for the selected assignment pipeline.

Also, I used data-column alignment in the preprocessing phase, in `expectation_maximization.py` (line 62). I did this because the Endorisk dataset contains some naming differences from the node names in the network file. Making sure these matched beforehand ensured that the EM algorithm did not crash due to variable mismatch (which happened many times before).

3 Implementation

3.1 Part 1: Variable Elimination

3.1.1 Goal and Mathematical Formulation

The goal of Part 1 was to implement exact inference using Variable Elimination, so the query using the Earthquake network is:

$$P(Alarm \mid Burglary = True) = \alpha \sum_{E,J,M} P(B, E, A, J, M)$$

Then I did the following:

1. Apply evidence by restricting factors (filtering DataFrame rows).
2. Multiply factors that share variables (using DataFrame inner merges).
3. Sum out non-query variables in a specific elimination order (using `groupby` and `sum`).
4. Normalize the final factor to get a valid distribution.

3.1.2 Complexity and Elimination Ordering

At first I hardcoded an elimination order, but then I implemented a **min-fill style heuristic** in the runner. A bad elimination order can explode intermediate factor sizes, changing the space complexity from very manageable to $\mathcal{O}(d^n)$. Min-fill: picks the variable whose elimination adds the fewest new connections between its neighbors. That usually keeps intermediate factors smaller. Least-degree: picks the variable with the fewest neighbors, so it tends to eliminate the

least connected node first. Fewest-factors: picks the variable that appears in the fewest factors, so it targets the variable with the smallest immediate impact on the factor set.

To select a specific heuristic, set the `VE_HEURISTIC` environment variable to `min_fill` (default), `least_degree`, or `fewest_factors` before running `run.py`.

3.1.3 Pseudocode

```
function VariableElimination(factors, query_var, evidence):
    # Restrict factors to evidence
    for each variable in evidence:
        factors = restrict(factors, variable, evidence[variable])

    # Eliminate non-query variables
    for each variable to eliminate (in order):
        # Collect all factors mentioning this variable
        relevant_factors = factors that mention variable
        other_factors = factors that don't mention variable

        # Multiply relevant factors
        product = multiply_all(relevant_factors)

        # Sum out the variable
        summed = marginalize(product, variable)

        # Update factor list
        factors = other_factors + [summed]

    # Multiply remaining factors and normalize
    result = multiply_all(factors)
    return normalize(result)
```

Listing 1: Variable Elimination Algorithm

3.2 Part 2: MAP Inference

3.2.1 Goal

Part 2 cranks Variable Elimination up to MAP (Maximum a Posteriori) inference, so it is zeroing in on the single most likely assignment of key variables, given the evidence in hand. Here's the current query setup:

- MAP variables: `Burglary`, `Earthquake`
- Evidence: `Alarm=True`

3.2.2 Implementation Details

I reused most of the VE infrastructure but added MAP-specific behaviour. Instead of summing out variables, we maximise over them. The tricky part is that computing the max probability loses the actual assignment. To solve this, I added an argmax system. During elimination, I

record which specific value yields the maximum probability, allowing me to reconstruct the final MAP assignment.

3.2.3 Pseudocode

```
function MAP(factors, map_vars, evidence):
    # Restrict factors to evidence
    for each variable in evidence:
        factors = restrict(factors, variable, evidence[variable])

    # Identify elimination order for non-MAP variables
    elim_order = build_elimination_order(factors, exclude=map_vars)

    # Eliminate non-MAP variables using maximization
    assignments = {}
    for each variable in elim_order:
        relevant_factors = factors that mention variable
        other_factors = factors that don't mention variable

        # Multiply relevant factors
        product = multiply_all(relevant_factors)

        # Maximize out the variable (instead of summing)
        max_factor, arg_assignment = maximize(product, variable)
        assignments[variable] = arg_assignment

        factors = other_factors + [max_factor]

    # Find MAP assignment for remaining variables
    result = multiply_all(factors)
    for each map_var in map_vars:
        # Backtrace to find the value that gave max probability
        max_assignment[map_var] = argmax_value(result, map_var)

    return max_assignment, max_probability
```

Listing 2: MAP (Maximum A Posteriori) Inference

3.3 Part 3: Expectation-Maximization (EM) Learning

3.3.1 Goal

In this part I have to implement EM on Endorisk-style data with hidden nodes, especially *Lymph node metastasis (LNM)* and *Myometrial invasion*. Because direct counting is impossible with missing data, EM iteratively guesses the missing values and updates the model with this newfound information.

3.3.2 Implementation Pipeline

I implemented EM with the following iterative steps

- **Random Initialization:** Generating starting parameters using restart-dependent seeds.

- **E-step:** Computing expected sufficient statistics via exact inference over the hidden variables for every row of data.
- **M-step:** Updating CPT parameter tables with Laplace smoothing to prevent zero-probability crashes.
- **Monitoring:** Tracking log-likelihood and parameter-change values per iteration.

3.3.3 Pseudocode

```
function EM(network, data, hidden_vars, max_iter, tol):
    best_network = None
    best_ll = -infinity

    # Try multiple random restarts
    for restart in 1 to num_restarts:
        # Random initialization
        theta = randomize_cpts(network, restart_seed)

        prev_ll = -infinity
        for iteration in 1 to max_iter:
            # E-step: compute expected counts
            expected_counts = {}
            total_ll = 0.0

            for each row in data:
                observed = observed_variables(row)

                # Infer hidden variables using Variable Elimination
                posterior = run_ve(network, theta, hidden_vars, observed)

                # Accumulate expected sufficient statistics
                for each hidden assignment in posterior:
                    prob = posterior[hidden_assignment]
                    full_assignment = observed + hidden_assignment
                    expected_counts += prob * count(full_assignment)
                    total_ll += log(evidence_probability)

            # M-step: update parameters using expected counts
            for each node in network:
                theta[node] = MLE_estimate(expected_counts[node]) + Laplace(
    alpha)

            # Check convergence
            if |total_ll - prev_ll| < tol:
                break
            prev_ll = total_ll

        # Track best model
        if total_ll > best_ll:
            best_ll = total_ll
            best_network = theta
```

```
42
43      return best_network, best_ll
```

Listing 3: Expectation-Maximization (EM) Algorithm

3.4 Implementation of the formative feedback

I used the formative feedback to add some things:

3.4.1 Feedback 1: Improve elimination ordering

- I replaced the fixed ordering with the min-fill style ordering.
- I reused this ordering idea in VE, MAP, and aligned the EM inference calls with it.

Observed Effect: The peak size of intermediate factors drastically decreased. Runtime became much more stable, avoiding out-of-memory errors on larger evidence sets.

3.4.2 Feedback 2: Improve informative logging

- I added detailed MAP logs: factor buckets, products, elimination/maximization steps, and backtrace traces.
- I added detailed EM logs: run settings, restart seeds, progress lines, per-iteration likelihood, and best run summaries.

Observed Effect: Debugging became much faster. Sensitive bugs felt more "real" and visible, and experiment comparison became as simple as reading `log_endorisk.txt`.

4 Results

As seen in the detailed execution logs and comparison output, the implemented algorithms produce the expected outputs for the test cases I checked.

4.1 Inference Benchmarks

- **Variable Elimination Validation:** The Variable Elimination engine correctly marginalizes over the Earthquake variable to calculate $P(Alarm = True \mid Burglary = True) = 0.9402$. It effectively hit the exact floating-point value.
- **MAP Backtrace Validation:** The MAP inference correctly identifies that when the Alarm rings, the most probable explanation is `Burglary=True` and `Earthquake=False` with a probability of 0.009212. The detailed backtrace confirms that leaf nodes without evidence (like `JohnCalls` and `MaryCalls`) are efficiently summed out to 1.0, acting as a massive optimization.

4.2 EM Optimizations and Plausibility

The EM implementation uses several important optimizations to make parameter learning with hidden variables more efficient.

Multi-Restart Strategy: The system runs EM five times with different fixed seeds (1337, 1354, 1371, 1388, 1405) to avoid getting stuck in local optima. In the latest run, the best log-likelihood was -507.0331 with seed 1337 after 10 iterations, showing similar convergence patterns for all restarts. Each run starts with random initialization, and the model with the highest final log-likelihood is kept as the learned network.

Dictionary-Based Hot Path: I improved the EM hot path by switching from DataFrame-based expected-count updates to using dictionaries for accumulation in the E-step and for building denominators in the M-step. This keeps the EM logic and results the same, but cuts down on pandas overhead in the main loop and makes each iteration much faster (with a measured speedup of 30-40

Variable Separation: The code organizes variables into three groups: (1) nodes directly linked to hidden variables LNM and MyometrialInvasion, which need EM updates; (2) nodes that are observed in every dataset row, which use direct MLE; and (3) partially observed nodes, which use EM with evidence conditioning.

Learned Model Characteristics: Comparing the learned model to the original Endorisk model shows clear changes in parameters. The best-learned CPT has mean probability changes for several variables: LNM has a mean of 0.322 and MyometrialInvasion has a mean of 0.370, reflecting the updated posterior distributions from the hidden variables. Other nodes, such as CA125 (mean 0.382) and Recurrence (mean 0.254), show how the hidden variables indirectly affect the rest of the network.

5 Conclusion

I developed an end-to-end Bayesian network pipeline that integrates three primary inference and learning components: Variable Elimination for exact probabilistic queries, MAP inference with backtracking for maximum a posteriori estimation, and the Expectation-Maximization (EM) algorithm for parameterThe most significant improvements resulted from two design choices: implementing heuristic-based elimination ordering, which reduced computational complexity from exponential to a tractable level, and incorporating extensive logging and parameter tracking, which enhanced transparency and facilitated debugging. The final trained Endorisk network demonstrates interpretable parameter shifts that align with the inferred posteriors of latent variables, providing evidence for the correctness of the EM procedure on incomplete medical datasets.e on incomplete medical datasets.